

Разработка драйверов в Linux, 2

Владислав Богданов

2026-06-16

Оглавление

1. Пространство ядра	1
2. Параметры модуля	4
2.1. module_param(m_count, int, S_IRUSR S_IWUSR S_IRGRP S_IWGRP S_IOTH).....	4
2.2. MODULE_PARM_DESC(m_count, "module_counter");	6
2.3. Полный пример	6
2.4. Передача параметра в модуль	7
2.5. Где посмотреть	7

Материал написан на основе курса "Разработка модулей ядра Linux (Linux Kernel developer)" от <https://inzhenerka.tech/> и собранных данных в интернете.

1. Пространство ядра

Symbol - идентификатор функции.

Kernel symbol - имена функций которые может использовать любой модуль ядра, обращаться к ним и вызывать их.

Список доступных имен

```
cat /proc/kallsymbol
```

К примеру `cat /proc/kallsyms | grep printk` выведет огромное количество имен. То есть когда мы используем `printk` мы используем не функцию из библиотеки а команду имя из модуля. То есть в ядре модуль отвечающий за логирование из ядра.

При компиляции **obj-m** указывает список объектников из которых собираются модули.

```
CURRENT = $(shell uname -r)
KERNEL_DIR = /lib/modules/$(CURRENT)/build
PWD = $(shell pwd)

obj-m := hello.o my_alert.o

hello-objs := start.o stop.o my_number.o my_time.o create_file.o

default:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean
```

Для того что бы код стал элементом пространства имен ядра нужно подать команду макрос после объявления функции `EXPORT_SYMBOL(my_alert)`.

```
#include "linux/module.h"

MODULE_LICENSE("GPL")
MODULE_AUTHOR("user")

int my_alert(const char *dev, const char *msg)
{
    printk(KERN_ALERT DEV_NAME "Module %s sent message: %s\n", dev, msg);
}
```

```

    return 0;
}

EXPORT_SYMBOL(my_alert)

```

В модуле отсутствует *init* и *cleanup* это означает, что ядру не нужно выполнять никакой логики при загрузке и выгрузке модуля, **но если кто то из них есть должен быть и второй.**

```

sudo insmod ./my_alert.ko
sudo cat /proc/kallsyms | grep my_alert
ffffffffc0c5f068 r __kstrtab_my_alert [my_alert] // строковое название
экспортированного символа это нужно что бы мы могли явно обращаться из других программ
и знали бы это имя
ffffffffc0c5f071 r __kstrtabns_my_alert [my_alert]
ffffffffc0c5f000 r __ksymtab_my_alert [my_alert] // структура содержащие подробное
описание экспортированной конструкции (например адрес памяти, вх вых параметры и тд)
ffffffffc0c5f00c r __crc_my_alert [my_alert]
ffffffffc0c5f074 r __UNIQUE_ID_note_317 [my_alert]
ffffffffc0c5f08c r __UNIQUE_ID_note_316 [my_alert]
ffffffffc0a7c010 T my_alert [my_alert]
ffffffffc0c5d040 d __this_module [my_alert]
ffffffffc0a7c000 t __pfx_my_alert [my_alert]
// угловые скобки показывают из какого модуля

```

Имена функций должны включать название модуля, для того что бы не было совпадения с другими функциям то есть:

```

hello1 hello1_my_alert()
hello2 hello2_my_alert()

```

Выгрузить модуль, который используется другим не получится.

Модуль вызывающий функцию `my_alert`.

```

//hello.h
#ifndef _HELLO_H_
#define _HELLO_H_

#include <linux/module.h>
#include <linux/types.h>

/*device name*/
#define DEV_NAME "hello"

/*extern - не обязателен, так как функции и так объявляются как внешние*/
int my_alert(const char *dev, const char *msg);

```

```

#endif /*_HELLO_H_*/

//start.c
#include "hello.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("user");
MODULE_DESCRIPTION ("hello module");

int init_module(void)
{
    my_alert(DEV_NAME, " started");
    return 0;
}

// stop.c
#include "hello.h"

void cleanup_module(void)
{
    my_alert(DEV_NAME, " exited");
}

```

В каталоге с скомпилированным модулем ядра файл **Module.symvers**, содержимое данного файла показывает список символов модуля.

```

cat Module.symvers
0xe43f5fdd my_alert my_alert EXPORT_SYMBOL

```

Если мы ссылаемся на модуль в другом каталоге нужно указать путь к Module.symvers

```

KBUILD_EXTRA_SYMBOLS = ../my_alert/Module.symvers

CURRENT = $(shell uname -r)
KERNEL_DIR = /lib/modules/$(CURRENT)/build
PWD = $(shell pwd)

obj-m := hello.o

hello-objs := start.o stop.o my_number.o my_time.o create_file.o

default:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean

```

2. Параметры модуля

2.1. `module_param(m_count, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IOTH)`

Параметры модуля обычно глобальны. Если нужны только для запуска обычно они `static`. Добавляются два макроса один задает параметр, другой его описание.

В ядре Linux для `module_param` есть целый набор predefined типов. Они определены в заголовочном файле `<linux/moduleparam.h>`. Разберём их по группам.

Тип в <code>module_param</code>	Соответствующий тип C	Что хранит
<code>byte</code>	<code>unsigned char</code>	1 байт (0-255)
<code>short</code>	<code>short</code>	Знаковое 16-битное
<code>ushort</code>	<code>unsigned short</code>	Беззнаковое 16-битное
<code>int</code>	<code>int</code>	Знаковое 32-битное
<code>uint</code>	<code>unsigned int</code>	Беззнаковое 32-битное
<code>long</code>	<code>long</code>	Знаковое (32 или 64 бита)
<code>ulong</code>	<code>unsigned long</code>	Беззнаковое (32 или 64 бита)
<code>llong</code>	<code>long long</code>	Знаковое 64-битное
<code>ullong</code>	<code>unsigned long long</code>	Беззнаковое 64-битное
<code>bool</code>	<code>bool</code>	<code>true</code> / <code>false</code> (или Y/N, 1/0)
<code>invbool</code>	<code>bool</code>	Инвертированный <code>bool</code> (Y → <code>false</code>)
<code>charp</code>	<code>char *</code>	Строка

`module_param(m_count, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IOTH);`

- `m_count` - переменная
- `int` - тип данных, NOTE: Когда передаешь строку при загрузке модуля (например: `insmod my_module.ko m_char="Hello Kernel"`), ядро само выделяет память под эту строку в пространстве ядра. Из-за этого есть строгое правило: Никогда не вызывай `kfree(m_char)` в функции выгрузки модуля (`module_exit`). Ядро само освободит эту память при выгрузке модуля. Если ты попытаешься сделать `kfree` вручную, ты получишь `kernel panic` (ошибку `double-free` или повреждение структур ядра).
- `S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IOTH` Эти макросы взяты из стандартных заголовков POSIX (определены в `<linux/stat.h>`) и описывают стандартные права доступа к файлам в Unix-подобных системах.

Расшифровка флагов, аббревиатуры расшифровываются так:

I = Inode (исторически)

R = Read (чтение)

W = Write (запись)

X = Execute (выполнение, здесь не используется, так как это не исполняемый файл)

USR = User (владелец файла)

GRP = Group (группа владельца)

OTH = Others (все остальные пользователи)

то есть:

1. S_IRUSR — Read для User (Владелец может читать параметр).
2. S_IWUSR — Write для User (Владелец может изменять параметр).
3. S_IRGRP — Read для Group (Группа может читать).
4. S_IWGRP — Write для Group (Группа может изменять).
5. S_IROTH (вместо S_IOTH) — Read для Others (Все остальные могут читать).

Когда загружаешь модуль с таким `module_param`, ядро автоматически создает виртуальный файл в файловой системе `sysfs`. Путь к нему будет примерно такой: `/sys/module/<имя_твоего_модуля>/parameters/m_count`. Права доступа к этому файлу будут соответствовать флагам. Если перевести эти макросы в привычную восьмеричную (`octal`) систему счисления, которую так любят линуксоиды, то получится:

- User: `Read(4) + Write(2) = 6`
- Group: `Read(4) + Write(2) = 6`
- Others: `Read(4) = 4`
- Итого: `0664` (или просто `664`)

В реальном коде ядра Linux разработчики редко пишут такие длинные конструкции с макросами. Обычно используют короткую восьмеричную запись.

```
module_param(m_count, int, 0664);
```

Давать право на запись (W) группе и всем остальным (OTH) в параметры ядра — **очень плохая практика**. Параметры ядра управляют поведением самой сердцевины ОС. Если обычный пользователь сможет изменить `m_count` (или, что хуже, какой-нибудь указатель или размер буфера), это может привести к падению ядра (Kernel Panic) или уязвимости.

Как делают правильно: Обычно параметры ядра делают доступными на запись только для `root` (владельца), а остальным дают только чтение. Для этого используют права `0644` (или макрос `S_IRUGO` | `S_IWUSR`, где `UGO` = User/Group/Others).

```
// Правильный и безопасный вариант:
```

```
module_param(m_count, int, 0644);
```

В этом случае root сможет делать `echo 10 > /sys/module/.../parameters/m_count`, а обычные пользователи смогут только `cat /sys/module/.../parameters/m_count`. Если параметр вообще не должен меняться после загрузки модуля, используют 0444 (только чтение для всех).

2.2. MODULE_PARM_DESC(m_count, "module_counter");

Когда пишешь модуль, другие разработчики (или ты сам через месяц) могут не знать, что означает `m_count` — это счётчик? максимальное количество? номер модуля? Описание отвечает на эти вопросы.

Где это описание можно увидеть

1. Через modinfo

```
modinfo my_module.ko

filename:      /lib/modules/5.15.0/extra/my_module.ko
license:      GPL
description:   My test module
author:       Your Name
parm:         m_count:module_counter (int)
parm:         m_char:Строковый параметр для передачи текста в модуль (charp)
```

2. Через sysfs (если параметр доступен)

```
cat /sys/module/my_module/parameters/m_count
```

2.3. Полный пример

```
#include <linux/module.h>

static int m_count = 5;
static char *m_char;

module_param(m_count, int, 0644);
MODULE_PARM_DESC(m_count, "Количество циклов обработки при загрузке модуля (по умолчанию: 5)");

module_param(m_char, charp, 0644);
MODULE_PARM_DESC(m_char, "Имя устройства для инициализации (например: 'ttyS0', 'eth0')");
```



```
static int __init my_init(void)
{
    printk(KERN_INFO "Запуск с m_count=%d, m_char=%s\n",
           m_count, m_char ? m_char : "не задано");
    return 0;
}

module_init(my_init);
MODULE_LICENSE("GPL");
```

2.4. Передача параметра в модуль

```
// один параметр
sudo insmod param_demo1.ko m_count=3

// строки к примеру из двух слов, должно быть без пробелов у названия параметра и у
кавычек
sudo insmod param_demo1.ko m_char='"qwe rty"'

// передача нескольких параметров
sudo insmod param_demo1.ko m_count=5 m_char="Hello Kernel"
```

Если у тебя есть параметр-массив (через `module_param_array`), значения перечисляются через запятую:

```
// В коде модуля:
static int ports[5];
static int ports_count;
module_param_array(ports, int, &ports_count, 0644);
```

```
# Передача массива:
sudo insmod my_module.ko ports=80,443,8080,8443
# ports_count автоматически станет равен 4
```

2.5. Где посмотреть

в директориях:

- `/proc/module;`
- `/sys/module.`

```
user@ubuntu:~/prj/linux/param_demo1$ cd /sys/module/par
param_demo1/ parport/      parport_pc/
user@ubuntu:~/prj/linux/param_demo1$ cd /sys/module/param_demo1/
```

```
user@ubuntu:/sys/module/param_demo1$ ls
coresize holders initsize initstate notes parameters refcnt sections
srcversion taint uevent
user@ubuntu:/sys/module/param_demo1$ cd parameters/
user@ubuntu:/sys/module/param_demo1/parameters$ ls
m_char m_count
user@ubuntu:/sys/module/param_demo1/parameters$ cat m_char
Linux
user@ubuntu:/sys/module/param_demo1/parameters$ cat m_count
3
user@ubuntu:/sys/module/param_demo1/parameters$
```

Домашнее задание:

- модифицировать простой модуль что бы он экспортировал какой нибуть полезный символ, время или случайное число и воспользоваться этим символом в другом модуле.
- передать модуль диапазон для генерации случайного числа (по умолчанию от а до б).